

Adaptive Virtual Memory Management for LLM Inference Serving on AI Chips

Abstract

Deploying large language models (LLMs) for real-time use, such as in chatbots and AI assistants, places heavy demands on the memory systems of the AI chips that run them. Prior research on Ascend NPUs has developed virtual memory management (VMM) strategies to reduce memory fragmentation during model training, achieving measurable gains in memory efficiency. However, these strategies have not been examined in the context of inference serving, where memory behavior is markedly different: instead of a few large, predictable allocations, an inference server must continuously create and release small memory blocks for each user's key-value (KV) cache as responses are generated in real time. This paper investigates whether training-optimized memory allocators remain effective under these conditions, and proposes a new memory management approach built specifically for inference workloads. The proposed system introduces a page-based memory reuse strategy suited to NPU virtual memory primitives, paired with a self-adjusting allocation policy that reacts to live traffic patterns rather than relying on fixed, pre-tuned constants. The system is evaluated on Ascend NPU hardware using representative open-source LLMs under simulated multi-user serving conditions with varying request lengths, arrival patterns, and batch sizes. The study measures memory utilization, fragmentation, allocation overhead, and end-to-end response latency to determine whether the proposed design offers meaningful improvements over both the native allocator and the existing training-oriented approach. This work aims to demonstrate that memory management strategies must be tailored to the specific demands of inference serving rather than repurposed from training-focused designs.

Research Gap

Existing NPU virtual memory management work, including fusion-pool-style allocators, was designed and validated for training workloads, where allocations are large, few in number, and highly predictable across iterations. Inference serving for LLMs breaks all three assumptions: KV-cache memory is allocated and freed continuously in small, per-user chunks, request lengths and arrival patterns vary unpredictably in real time, and allocator decisions sit directly on the latency-critical path of every generated token. No prior study has tested whether a training-oriented allocator even remains effective once these conditions are introduced, and no NPU-native allocator has been purpose-built around the page-level reuse patterns that inference workloads actually produce. This paper addresses that gap by (i) evaluating a training-optimized allocator under inference conditions for the first time, and (ii) proposing a page-based, traffic-adaptive allocator designed specifically for the fragmentation and latency demands of real-time LLM serving.

Methodology

- 1. Environment Preparation** — Set up the Ascend NPU hardware environment, including the required chip firmware, runtime libraries, and the PyTorch NPU extension needed to run and modify the memory allocator.
- 2. Baseline Systems Selection** — Identify systems to compare against: the unmodified, native Ascend memory allocator; the existing training-oriented fusion-pool allocator, tested here for the first time under inference conditions; and, optionally, a GPU-based reference system such as vLLM's PagedAttention for cross-platform context.
- 3. Workload Simulation Design** — Build synthetic inference workloads that reflect real chatbot-style usage: requests with short, medium, and long input/output lengths; steady traffic versus sudden bursts of requests; and fixed-size batching versus continuous, rolling batching.
- 4. System Development** — Implement the proposed memory manager: a page-based allocation layer for KV-cache memory built on NPU-native virtual memory functions, an adaptive controller that adjusts allocation thresholds automatically based on recent traffic behavior, and logging instrumentation to record every memory event for later analysis.
- 5. Running the Experiments** — Execute each baseline and the proposed system against the same set of workloads, repeating runs to reduce measurement noise.
- 6. Metric Collection** — Measure and record memory utilization ratio, fragmentation ratio, allocator-level operation latency, time-to-first-response and per-token response latency, and overall throughput in requests and tokens processed per second.
- 7. Comparative Analysis** — Compare results across all systems and workload types, and separately test the effect of the adaptive controller versus fixed thresholds.
- 8. Interpretation and Reporting** — Analyze patterns in the results, identify where and why improvements occur, and document any trade-offs, such as added computational overhead.

Workflow Diagram

The diagram below summarizes the end-to-end experimental workflow, from environment setup through interpretation and reporting, with the core experiment-and-measure cycle shown as a repeating loop.

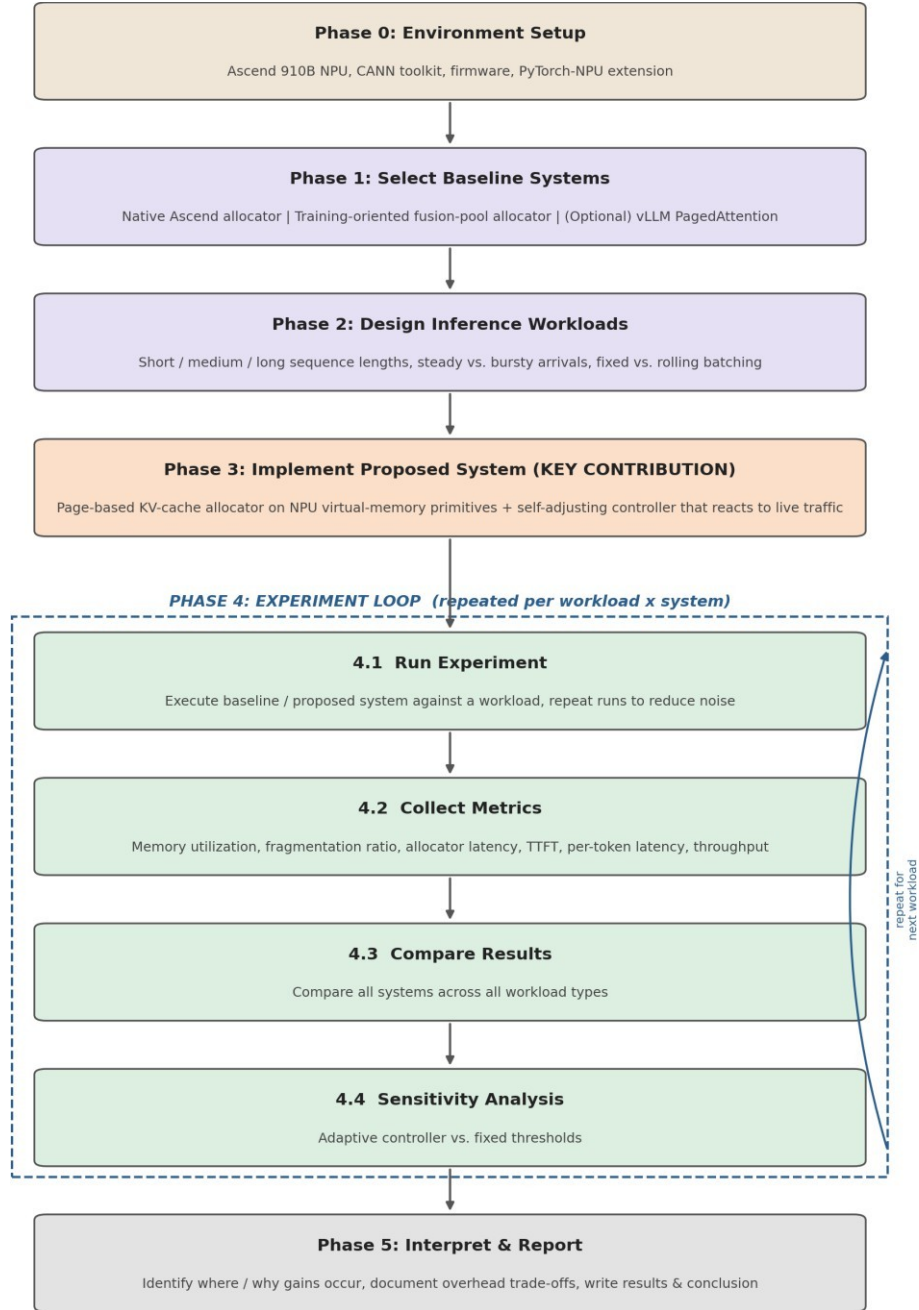


Fig. 1. Experimental workflow from setup through analysis.

Research Contributions

- First empirical evaluation of a training-oriented NPU allocator (fusion-pool) under real-time, multi-user LLM inference conditions.
- A page-based KV-cache reuse strategy built directly on NPU-native virtual memory primitives, rather than adapted from GPU or training-era designs.

- A self-adjusting allocation policy that reacts to live traffic patterns instead of relying on fixed, pre-tuned constants.
- A unified evaluation of memory utilization, fragmentation, allocator overhead, and end-to-end latency across native, training-oriented, and proposed allocators on the same NPU hardware.
- A sensitivity analysis isolating the effect of the adaptive controller versus fixed thresholds, clarifying where the adaptivity itself contributes value.